

## **Software/ML Domain Task-6**

Name: Unnabh Baruah

Reg. no: 240961174

Domain: Software/ML

Repository: [github.com/big2ooth/Synergy\\_TP](https://github.com/big2ooth/Synergy_TP)

Date: 27/06/2026

## **Abstract**

This report covers the concepts behind tasks 1 to 5 completed during the Software/ML Taskphase. The work spanned setting up a version-controlled repository, writing Python scripts for file handling, building a CSV parser from scratch, cleaning a messy dataset using pandas and generating visualizations with matplotlib. This report explains the theory and reasoning behind each step rather than describing the code itself.

## **Introduction**

Tasks 1 through 5 cover a full pipeline: from setting up a project repository to producing visual summaries of data. The goal of this report is to explain the concepts that make each task work, the decisions made during implementation and what was learned at each stage.

## **Development Environment and Version Control**

### **Git and GitHub**

Git is a version control system that tracks changes to files over time. Every change is saved as a commit, which includes a message describing what changed. This makes it easy to go back to an earlier version if something breaks. GitHub is a platform that hosts Git repositories online, making it possible to share code and collaborate. While working on these tasks very basic git commands like clone, diff, restore, add, commit and push were implemented.

### **Virtual Environments and Dependencies**

A virtual environment is an isolated Python installation for a specific project. It prevents packages installed for one project from interfering with another. The command 'python -m venv venv' creates one and it is activated before installing any packages using the command 'source venv/Scripts/activate'(for gitbash). All required packages are listed in requirements.txt so anyone can reproduce the environment by running 'pip install -r requirements.txt'. The .gitignore file tells Git to skip certain files and folders, such as the venv folder and \_\_pycache\_\_, which do not need to be committed since they can be recreated.

## **Python and File Handling Concepts**

Across all tasks, a consistent set of Python features was used. Functions were written for every distinct operation so that each piece of logic could be tested and reused independently. Lists and dictionaries were the main data structures: lists held rows of data and dictionaries mapped column names to values. File I/O was done using the `open()` function with context managers, which automatically close the file after reading or writing. Type hints such as `'list[dict]'` and `'str'` were added to function signatures to make the expected inputs and outputs clear. Try/except blocks were used to handle cases like a score that could not be converted to a number, so the program would skip the bad value and continue rather than crash.

## **CSV Parsing and Pandas**

### **Manual CSV parsing**

A CSV file is a plain text file where each line is a row and values are separated by commas. The first line is usually the header, which names each column. In Task 3, the CSV was read manually using `open()` and a for loop. The header was split on commas to get column names, and each following line was split the same way to get values. These were zipped together into a dictionary representing one row. This approach makes the mechanics of CSV parsing visible: it is just string splitting and type conversion. Empty lines and rows with the wrong number of values were skipped with a simple length check.

### **Pandas-based loading**

Pandas is a Python library that handles all of this automatically. `pd.read_csv()` reads a file, infers types, and returns a `DataFrame`, which is a table with named columns. The same summary statistics computed manually were replicated using `groupby()`, `idxmax()`, and `value_counts()`. Comparing both outputs confirmed that the manual parser produced the same results, which validates that the manual logic was correct.

## **Data Cleaning**

### **Why cleaning is necessary**

Real-world data is rarely clean. Task 4 used a dataset with duplicate rows, inconsistent category names, numbers stored as words, heights in different units, and missing values. Using this data

directly would produce wrong results. Cleaning means bringing all values into a consistent, usable format before any analysis.

## **Cleaning decisions**

Each type of issue required a different approach. Duplicate rows were dropped using `drop_duplicates()`. Domain names like 'ml', 'MACHINE LEARNING', and 'Web Dev' were mapped to a fixed set of canonical labels using a dictionary. Attendance values were stripped of their % symbol and converted to floats. Rows with attendance below 0 or above 100 were removed entirely because replacing them with an estimate would introduce fabricated data. Word-based numbers like 'nine' and 'two' were converted using a lookup dictionary. Heights in metres were multiplied by 100 to give centimetres. Missing numeric values were filled with the column median, which is more stable than the mean when there are outliers.

## **Validation**

After cleaning, nine checks were run on the resulting dataframe. These confirmed that there were no duplicate student IDs, all numeric columns contained only numbers, attendance was within the valid range, submitted contained only yes or no, domain contained only the four allowed values, and no critical column had any null values. Validation is important because it catches cases where a cleaning step silently failed or produced an unexpected result.

## **Visualization**

Task 5 used matplotlib to generate three plots from the cleaned dataset. Each plot was saved directly to a file using `savefig()` without opening a window, which is important for scripts that run without a display. `tight_layout()` was called before saving to prevent axis labels from being cut off.

### **Plot 1: Average score by domain**

This bar chart groups students by domain and shows the average score for each group. It makes it easy to see at a glance which domain performed better. The x-axis shows domain names and the y-axis shows the average score.

### **Plot 2: Attendance vs score**

This scatter plot places each student as a point, with attendance on the x-axis and score on the y-axis. It helps identify whether students who attended more tended to score higher. With only eight data points the trend is not conclusive, but the chart makes the relationship visible.

### Plot 3: Submission status count

This bar chart shows how many students submitted the task versus how many did not. It gives a quick count of completion rates without needing to read through the raw data.

## Data Summary

| Task   | Topic            | Main Concept Learned                                                                            |
|--------|------------------|-------------------------------------------------------------------------------------------------|
| Task 1 | Git and GitHub   | Setting up a repository, writing commits, and keeping a clean version history.                  |
| Task 2 | Python scripting | Writing reusable functions, reading files, and handling exceptions.                             |
| Task 3 | CSV parsing      | How raw CSV text is read manually versus using pandas, and how both produce the same results.   |
| Task 4 | Data cleaning    | Handling missing values, fixing inconsistent data, converting units, and validating the result. |
| Task 5 | Visualization    | Turning cleaned tabular data into bar charts and scatter plots using matplotlib.                |

Table 1: Summary of Tasks 1 to 5 and the main concept from each.



Figure 1: Workflow from raw CSV data to cleaned dataset to visualized output.

## Conclusion

Working through Tasks 1 to 5 gave a clear picture of how data moves from raw form to something useful. Git kept the work organized and recoverable. Python file I/O showed what happens at the lowest level when reading structured text. Pandas made the same operations faster and less error-prone. Data cleaning required thinking carefully about each issue and deciding whether to remove, convert, or impute a value. Finally, matplotlib turned numbers into charts that are easier to read than a table of values. Together these skills form the basic pipeline used in almost any data-related project.

## References

1. Python Documentation: <https://docs.python.org/3/>
2. pandas Documentation: <https://pandas.pydata.org/docs/>
3. matplotlib Documentation: <https://matplotlib.org/stable/>
4. Git Documentation: <https://git-scm.com/doc>
5. Python csv module: <https://docs.python.org/3/library/csv.html>